

Programming Language Principles, Design, and Implementation

Lecture 8 - Type Inference

University of Birmingham

Where are we?

- ▶ **Part 1: Principles of programming**
 - ▶ Functional languages
 - ▶ Operational semantics
 - ▶ **Type systems**
 - ▶ Abstract datatypes
- ▶ Part 2: Compilers

Today

Type inference

- ▶ type checking vs. type inference
- ▶ Wand's type inference algorithm
- ▶ Robinson's unification algorithm
- ▶ principal types

Further reading:

- ▶ “Types and Programming Languages”, Pierce, Chapter 22

Recap: the Simply Typed λ -Calculus

Types are useful in many respects:

- ▶ provide guarantees on how terms compute
- ▶ in particular that typed expressions **do not get stuck**
- ▶ document the code
- ▶ used for optimizations (e.g., memory usage)

The **Simply Typed λ -Calculus** is one of the most elementary type systems.

Syntax (Church-style):

$$T ::= \mathbb{B} \mid T \rightarrow T$$
$$M ::= x \mid \lambda x : T. M \mid M M \mid \text{true} \mid \text{false} \mid \text{if } M \text{ then } M \text{ else } M$$

Recap: Typing Rules

The typing rules of the Church-style Simply Typed λ -Calculus
(syntax directed):

$$\frac{}{\Gamma, x : T \vdash x : T} \text{VAR}$$

$$\frac{\Gamma, x : T \vdash M : U}{\Gamma \vdash \lambda x : T. M : T \rightarrow U} \text{ABS}$$

$$\frac{\Gamma \vdash M : T \rightarrow U \quad \Gamma \vdash N : T}{\Gamma \vdash M N : U} \text{APP}$$

$$\frac{}{\Gamma \vdash \text{true} : \mathbb{B}} \text{T}$$

$$\frac{}{\Gamma \vdash \text{false} : \mathbb{B}} \text{F}$$

$$\frac{\Gamma \vdash M : \mathbb{B} \quad \Gamma \vdash N : T \quad \Gamma \vdash P : T}{\Gamma \vdash \text{if } M \text{ then } N \text{ else } P : T} \text{ITE}$$

Type Checking vs. Type Inference

The typing rules of the Simply Typed λ -Calculus allow checking that expressions are well-typed.

Type checking: process of checking that some given **expression** has some given **type** w.r.t. given **typing rules**

- ▶ This problem presupposes that we are given a type
- ▶ λ -abstraction are annotated with types

As writing down these types can be tedious and brittle, could they instead be automatically inferred?

Type inference: process of inferring the type(s) of an expression without type annotations so that it type checks according to given typing rules.

Could we start from the **Curry-style Simply Typed λ -calculus** and infer types?

Type Checking vs. Type Inference

A bit of history...

The problem of inferring a typed λ -term M from a corresponding untyped λ -term N such that $\text{erase}(M) = N$ was first formulated by **Curry** (probably in the 30s) and by **Hindley** in 1969.

Milner then designed a type inference algorithm for Edinburgh LCF's ML programming language, for which he proved a soundness result in 1978.

The correctness of that algorithm was proved by **Damas** in 1982, who showed among other things that the algorithm finds **the most general types possible** (the **principal type**).

Wand later formally showed in 1987, that type inference can be reduced to unification by introducing another **constraint-based algorithm**.

This will be the main focus of this lecture.

Type Checking vs. Type Inference

Recall the (syntax directed) rules of the Curry-style Simply-Typed λ -Calculus:

$$\frac{}{\Gamma, x : T \vdash x : T} \text{VAR} \qquad \frac{\Gamma, x : T \vdash M : U}{\Gamma \vdash \lambda x.M : T \rightarrow U} \text{ABS}$$

$$\frac{\Gamma \vdash M : T \rightarrow U \quad \Gamma \vdash N : T}{\Gamma \vdash M N : U} \text{APP}$$

$$\frac{}{\Gamma \vdash \text{true} : \mathbb{B}} \text{T} \qquad \frac{}{\Gamma \vdash \text{false} : \mathbb{B}} \text{F}$$

$$\frac{\Gamma \vdash M : \mathbb{B} \quad \Gamma \vdash N : T \quad \Gamma \vdash P : T}{\Gamma \vdash \text{if } M \text{ then } N \text{ else } P : T} \text{ITE}$$

Can we use these rules for type inference?

Problem: without type annotations, the ABS rule does not tell us how to pick x 's type T .

To infer that type we need to know how x is used, i.e., we need to inspect M

Constraint-based Type Inference

Instead of trying to guess the type of λ -abstractions' arguments, often type inference algorithms generate **type variables** that are later refined when more information is available.

Constraint-based type inference algorithm, such as Wand's, split the process of reconstructing types into 2 parts:

- ▶ 1st phase: generate type constraints involving **type variables** in a syntax-directed way
- ▶ 2nd phase: solve the set of generated constraints

This delays the decision of picking a type of arguments of λ -abstractions to until the entire expression has been analysed so that the necessary information to make this decision is available.

Type, type variables, and type constraints:

- ▶ we extend the syntax of types with type variables of the form α :
$$T ::= \alpha \mid \mathbb{B} \mid T \rightarrow T$$
- ▶ type constraints are equalities of the form $T_1 = T_2$
- ▶ we use C to denote sets of type constraints

Wand's algorithm – 1st phase

Interface & state:

- ▶ **input:** a term
- ▶ **state:**
 - ▶ a set of type constraints C
 - ▶ a set of goals G of the form $\langle \Gamma; M; T \rangle$
- ▶ **output:** C

Algorithm:

- ▶ **input:** a term M_0 without type annotations
- ▶ **initialization:**
 - ▶ Set $C = \emptyset$ (empty set of type constraints)
 - ▶ set $G = \{\langle \Gamma_0; M_0; \alpha_0 \rangle\}$ (singleton set of goals), where
 - ▶ α_0 is a fresh type variable
 - ▶ Γ_0 maps M_0 's free variables to distinct fresh type variables
- ▶ **loop step:**
 - ▶ if $G = \emptyset$ return C
 - ▶ else
 - ▶ choose a goal $\langle \Gamma; M; T \rangle$ in G and remove it from G
 - ▶ apply the **action table** to $\langle \Gamma; M; T \rangle$
 - ▶ it returns new type constraints and goals: add them to C and G

Using the result of Wand's algorithm

Given a term M_0 this algorithm

- ▶ returns a set of type constraints C
- ▶ where α_0 stands for M_0 's type

Solve C

- ▶ if solvable
 - ▶ the solver produces a substitution S (maps type variables to types)
 - ▶ such that $\Gamma_0[S] \vdash M_0 : \alpha_0[S]$
- ▶ if not solvable, M_0 is not well-typed

This also holds about intermediary sub-goals too.

How do we apply type substitution? $X[S]$, where

$S = \alpha_1 \mapsto T_1, \dots, \alpha_n \mapsto T_n$ replaces all occurrences of α_i by T_i in X

For example: $(\alpha_1 \rightarrow \alpha_2)[\alpha_1 \mapsto \mathbb{B}, \alpha_2 \mapsto \mathbb{B} \rightarrow \mathbb{B}] = \mathbb{B} \rightarrow (\mathbb{B} \rightarrow \mathbb{B})$

If α occurs several times in a substitution, the last one (farthest to the right) “shadows” the earlier ones.

For example: $\alpha[\alpha \mapsto \mathbb{B}, \alpha \mapsto \mathbb{B} \rightarrow \mathbb{B}] = \mathbb{B} \rightarrow \mathbb{B}$

Action Table

The action table

- ▶ take a goal of the form $\langle \Gamma; M; T \rangle$
- ▶ and returns a pair of a set of constraints C and a set of sub-goals G

The actions rules:

- ▶ $\text{action}(\langle \Gamma; x; T \rangle) = (\{T = \Gamma(x)\}, \emptyset)$
- ▶ $\text{action}(\langle \Gamma; \lambda x.M; T \rangle) = (\{T = \alpha_1 \rightarrow \alpha_2\}, \{\langle \Gamma, x : \alpha_1 \rangle; M; \alpha_2 \rangle\})$
where α_1 and α_2 are fresh type variables
- ▶ $\text{action}(\langle \Gamma; M \ N; T \rangle) = (\emptyset, \{\langle \Gamma; M; \alpha \rightarrow T \rangle, \langle \Gamma; N; \alpha \rangle\})$
where α is a fresh type variable
- ▶ $\text{action}(\langle \Gamma; \text{true}; T \rangle) = (\{T = \mathbb{B}\}, \emptyset)$
- ▶ $\text{action}(\langle \Gamma; \text{false}; T \rangle) = (\{T = \mathbb{B}\}, \emptyset)$
- ▶ $\text{action}(\langle \Gamma; \text{if } M \text{ then } N \text{ else } P; T \rangle) = (\emptyset, \{\langle \Gamma; M; \mathbb{B} \rangle, \langle \Gamma; N; T \rangle, \langle \Gamma; P; T \rangle\})$

These action rules closely follow the typing rules presented above

- ▶ they introduce fresh type variables for yet unknown types

Wand's algorithm – Example

Run this algorithm on $M_0 = \lambda x. \lambda y. \lambda z. (xz)(yz)$

- ▶ $C = \emptyset, G = \{\langle \cdot; M_0; \alpha_0 \rangle\}$
- ▶ $C = \{\alpha_0 = \alpha_1 \rightarrow \alpha_2\}, G = \{\langle x : \alpha_1; \lambda y. \lambda z. (xz)(yz); \alpha_2 \rangle\}$
- ▶ $C = \{\alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4\}, G = \{\langle x : \alpha_1, y : \alpha_3; \lambda z. (xz)(yz); \alpha_4 \rangle\}$
- ▶ $C = \{\alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6\},$
 $G = \{\langle x : \alpha_1, y : \alpha_3, z : \alpha_5; (xz)(yz); \alpha_6 \rangle\}$
- ▶ $C = \{\alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6\},$
 $G = \{\langle x : \alpha_1, y : \alpha_3, z : \alpha_5; xz; \alpha_7 \rightarrow \alpha_6 \rangle, \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; yz; \alpha_7 \rangle\}$
- ▶ $C = \{\alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6\},$
 $G = \left\{ \begin{array}{l} \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; x; \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 \rangle, \\ \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; z; \alpha_8 \rangle, \\ \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; yz; \alpha_7 \rangle \end{array} \right\}$
- ▶ $C = \{\alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6\},$
 $G = \left\{ \begin{array}{l} \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; x; \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 \rangle, \\ \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; z; \alpha_8 \rangle, \\ \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; y; \alpha_9 \rightarrow \alpha_7 \rangle, \\ \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; z; \alpha_9 \rangle \end{array} \right\}$
- ▶ $C = \{\alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6, \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 = \alpha_1\},$
 $G = \left\{ \begin{array}{l} \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; z; \alpha_8 \rangle, \\ \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; y; \alpha_9 \rightarrow \alpha_7 \rangle, \\ \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; z; \alpha_9 \rangle \end{array} \right\}$
- ▶ $C = \{\alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6, \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 = \alpha_1, \alpha_8 = \alpha_5\},$
 $G = \{\langle x : \alpha_1, y : \alpha_3, z : \alpha_5; y; \alpha_9 \rightarrow \alpha_7 \rangle, \langle x : \alpha_1, y : \alpha_3, z : \alpha_5; z; \alpha_9 \rangle\}$
- ▶ $C = \left\{ \begin{array}{l} \alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6, \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 = \alpha_1, \\ \alpha_8 = \alpha_5, \alpha_9 \rightarrow \alpha_7 = \alpha_3 \end{array} \right\},$
 $G = \{\langle x : \alpha_1, y : \alpha_3, z : \alpha_5; z; \alpha_9 \rangle\}$
- ▶ $C = \left\{ \begin{array}{l} \alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6, \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 = \alpha_1, \\ \alpha_8 = \alpha_5, \alpha_9 \rightarrow \alpha_7 = \alpha_3, \alpha_9 = \alpha_5 \end{array} \right\},$
 $G = \emptyset$
- ▶ return C

Solving Constraints – Robinson's Unification Algorithm

Robinson's unification algorithm:

- ▶ take a set of type constraints C
- ▶ returns a **substitution** (of the form $\alpha_1 \mapsto T_1, \dots, \alpha_n \mapsto T_n$)

Algorithm:

- ▶ $\text{unify}(\emptyset) = \cdot$
- ▶ $\text{unify}(\{T = T\} \cup C) = \text{unify}(C)$
- ▶ $\text{unify}(\{\alpha = T\} \cup C) = \text{unify}(C[\alpha \mapsto T]) \circ (\alpha \mapsto T)$, where $\alpha \notin \text{FV}(T)$
- ▶ $\text{unify}(\{T = \alpha\} \cup C) = \text{unify}(C[\alpha \mapsto T]) \circ (\alpha \mapsto T)$, where $\alpha \notin \text{FV}(T)$
- ▶ $\text{unify}(\{T_1 \rightarrow T_2 = T_3 \rightarrow T_4\} \cup C) = \text{unify}(C \cup \{T_1 = T_3, T_2 = T_4\})$
- ▶ $\text{unify}(\{T_1 = T_2\} \cup C) = \text{ERROR}$, if no previous rules match

where

- ▶ **free type variables:**

$$\text{FV}(\alpha) = \{\alpha\}, \text{FV}(\mathbb{B}) = \emptyset, \text{FV}(T_1 \rightarrow T_2) = \text{FV}(T_1) \cup \text{FV}(T_2)$$

- ▶ **substitution composition** (an associative operation – to the left):
let $S_1 = \alpha_1 \mapsto T_1, \dots, \alpha_n \mapsto T_n$ and $S_2 = \alpha'_1 \mapsto U_1, \dots, \alpha'_m \mapsto U_m$
then: $S_1 \circ S_2 = S_1, \alpha'_1 \mapsto U_1[S_1], \dots, \alpha'_m \mapsto U_m[S_1]$

Robinson's Unification Algorithm – Example

Given $M_0 = \lambda x.\lambda y.\lambda z.(xz)(yz)$, Wand's algorithm returns:

$$C = \left\{ \begin{array}{l} \alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6, \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 = \alpha_1, \\ \alpha_8 = \alpha_5, \alpha_9 \rightarrow \alpha_7 = \alpha_3, \alpha_9 = \alpha_5 \end{array} \right\}$$

Apply Robinson's algorithm to this set

$$\begin{aligned}
& \triangleright \text{unify}\left(\left\{ \begin{array}{l} \alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6, \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 = \alpha_1, \\ \alpha_8 = \alpha_5, \alpha_9 \rightarrow \alpha_7 = \alpha_3, \alpha_9 = \alpha_5 \end{array} \right\}\right) \\
& \triangleright = \text{unify}\left(\left\{ \frac{\alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6, \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 = \alpha_1,}{\alpha_8 = \alpha_5, \alpha_9 \rightarrow \alpha_7 = \alpha_3, \alpha_9 = \alpha_5} \right\}\right) \circ (\alpha_0 \mapsto \alpha_1 \rightarrow \alpha_2) \\
& \triangleright = \text{unify}\left(\left\{ \frac{\alpha_4 = \alpha_5 \rightarrow \alpha_6, \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 = \alpha_1,}{\alpha_8 = \alpha_5, \alpha_9 \rightarrow \alpha_7 = \alpha_3, \alpha_9 = \alpha_5} \right\}\right) \circ (\alpha_2 \mapsto \alpha_3 \rightarrow \alpha_4) \circ (\alpha_0 \mapsto \alpha_1 \rightarrow \alpha_2) \\
& \triangleright = \text{unify}\left(\left\{ \frac{\alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 = \alpha_1,}{\alpha_8 = \alpha_5, \alpha_9 \rightarrow \alpha_7 = \alpha_3, \alpha_9 = \alpha_5} \right\}\right) \circ (\alpha_4 \mapsto \alpha_5 \rightarrow \alpha_6) \circ (\alpha_2 \mapsto \alpha_3 \rightarrow \alpha_4) \\
& \quad \circ (\alpha_0 \mapsto \alpha_1 \rightarrow \alpha_2) \\
& \triangleright = \text{unify}(\{\alpha_8 = \alpha_5, \alpha_9 \rightarrow \alpha_7 = \alpha_3, \alpha_9 = \alpha_5\}) \circ (\alpha_1 \mapsto \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6) \\
& \quad \circ (\alpha_4 \mapsto \alpha_5 \rightarrow \alpha_6) \\
& \quad \circ (\alpha_2 \mapsto \alpha_3 \rightarrow \alpha_4) \\
& \quad \circ (\alpha_0 \mapsto \alpha_1 \rightarrow \alpha_2) \\
& \triangleright = \text{unify}(\{\alpha_9 \rightarrow \alpha_7 = \alpha_3, \alpha_9 = \alpha_5\}) \circ (\alpha_8 \mapsto \alpha_5) \circ (\alpha_1 \mapsto \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6) \\
& \quad \circ (\alpha_4 \mapsto \alpha_5 \rightarrow \alpha_6) \\
& \quad \circ (\alpha_2 \mapsto \alpha_3 \rightarrow \alpha_4) \\
& \quad \circ (\alpha_0 \mapsto \alpha_1 \rightarrow \alpha_2) \\
& \triangleright = \text{unify}(\{\alpha_9 = \alpha_5\}) \circ (\alpha_3 \mapsto \alpha_9 \rightarrow \alpha_7) \circ (\alpha_8 \mapsto \alpha_5) \circ (\alpha_1 \mapsto \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6) \\
& \quad \circ (\alpha_4 \mapsto \alpha_5 \rightarrow \alpha_6) \\
& \quad \circ (\alpha_2 \mapsto \alpha_3 \rightarrow \alpha_4) \\
& \quad \circ (\alpha_0 \mapsto \alpha_1 \rightarrow \alpha_2) \\
& \triangleright = \frac{(\alpha_9 \mapsto \alpha_5) \circ (\alpha_3 \mapsto \alpha_9 \rightarrow \alpha_7)}{\alpha_4 \mapsto \alpha_5 \rightarrow \alpha_6} \circ (\alpha_8 \mapsto \alpha_5) \circ (\alpha_1 \mapsto \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6) \\
& \quad \circ (\alpha_2 \mapsto \alpha_3 \rightarrow \alpha_4) \\
& \quad \circ (\alpha_0 \mapsto \alpha_1 \rightarrow \alpha_2) \\
& \triangleright = \frac{(\alpha_9 \mapsto \alpha_5, \alpha_3 \mapsto \alpha_5 \rightarrow \alpha_7) \circ (\alpha_8 \mapsto \alpha_5)}{\alpha_4 \mapsto \alpha_5 \rightarrow \alpha_6} \circ (\alpha_1 \mapsto \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6) \\
& \quad \circ (\alpha_2 \mapsto \alpha_3 \rightarrow \alpha_4)
\end{aligned}$$

Principal Types

Given $M_0 = \lambda x. \lambda y. \lambda z. (xz)(yz)$, Wand's algorithm returns:

$$C = \left\{ \begin{array}{l} \alpha_0 = \alpha_1 \rightarrow \alpha_2, \alpha_2 = \alpha_3 \rightarrow \alpha_4, \alpha_4 = \alpha_5 \rightarrow \alpha_6, \alpha_8 \rightarrow \alpha_7 \rightarrow \alpha_6 = \alpha_1, \\ \alpha_8 = \alpha_5, \alpha_9 \rightarrow \alpha_7 = \alpha_3, \alpha_9 = \alpha_5 \end{array} \right\}$$

Using Robinson's algorithm, we obtain the following substitution:

$$S = \left(\begin{array}{l} \alpha_9 \mapsto \alpha_5, \alpha_3 \mapsto \alpha_5 \rightarrow \alpha_7, \alpha_8 \mapsto \alpha_5, \alpha_1 \mapsto \alpha_5 \rightarrow \alpha_7 \rightarrow \alpha_6, \alpha_4 \mapsto \alpha_5 \rightarrow \alpha_6, \\ \alpha_2 \mapsto (\alpha_5 \rightarrow \alpha_7) \rightarrow (\alpha_5 \rightarrow \alpha_6), \\ \alpha_0 \mapsto (\alpha_5 \rightarrow \alpha_7 \rightarrow \alpha_6) \rightarrow ((\alpha_5 \rightarrow \alpha_7) \rightarrow (\alpha_5 \rightarrow \alpha_6)) \end{array} \right)$$

Therefore, M_0 is typable:

- ▶ $\cdot[S] \vdash M_0 : \alpha_0[S]$
- ▶ i.e., $\cdot \vdash M_0 : (\alpha_5 \rightarrow \alpha_7 \rightarrow \alpha_6) \rightarrow ((\alpha_5 \rightarrow \alpha_7) \rightarrow (\alpha_5 \rightarrow \alpha_6))$
- ▶ this is M_0 's **principal type** (its most general type)

A **principal type** T for a closed term M is such that

- ▶ $\cdot \vdash M : T$
- ▶ if $\cdot \vdash M : T'$ then there exists a substitution S such that $T' = T[S]$

Conclusion

What did we cover today?

- ▶ Type inference for the Simply-Typed λ -Calculus
- ▶ Wand's type inference algorithm
- ▶ Robinson's unification algorithm
- ▶ Principal types

Next time?

- ▶ Polymorphism